

TalentLens AI: An Intelligent Resume Screening and Candidate Matching System Using Natural Language Processing and Large Language Models

Author Name

Department of Computer Science and Engineering

Institution Name

City, Country

email@example.com

Abstract—Recruitment workflows increasingly depend on digital applicant tracking systems, yet conventional resume screening remains limited by keyword matching, manual effort, inconsistent evaluation, and potential human bias. This paper presents TalentLens AI, an intelligent recruitment-assistance system that applies Natural Language Processing (NLP), semantic similarity, structured resume parsing, and Large Language Models (LLMs) to evaluate resumes against job descriptions. The proposed system extracts textual content from PDF, DOCX, and TXT resumes, identifies candidate information, technical skills, soft skills, professional experience, and job-specific gaps, and generates an Applicant Tracking System (ATS) compatibility score and job-match score. The system architecture consists of a React-based frontend, a Spring Boot REST backend, persistent evaluation storage, and AI API integration through Gemini-based LLM services. Unlike purely rule-based ATS tools, TalentLens AI combines lexical keyword overlap, entity extraction, contextual interpretation, and semantic reasoning to produce explainable recommendations for both recruiters and candidates. Results based on a realistic controlled evaluation scenario indicate that semantic AI-assisted screening can reduce manual review time while improving the interpretability of candidate-job alignment. The system is intended to support, rather than replace, human hiring decisions.

Index Terms—Artificial Intelligence, Natural Language Processing, Resume Screening, Applicant Tracking System, Semantic Similarity, Gemini API, Large Language Models, Recruitment Automation, React, Spring Boot

I. INTRODUCTION

Recruitment is one of the most data-intensive functions in modern organizations. A single job opening may attract hundreds or thousands of applications, making manual screening slow, expensive, and inconsistent. Traditional Applicant Tracking Systems (ATS) attempt to reduce this workload by filtering resumes according to keywords, formatting rules, degree requirements, and years of experience. Although these systems improve operational efficiency, they often fail to capture semantic meaning. For example, a candidate mentioning “customer relationship management” may be relevant to a role requiring “CRM experience,” even if the exact acronym is missing.

Recent developments in Natural Language Processing and Large Language Models have created new possibilities for

recruitment automation. Transformer-based models can represent text contextually, compare resumes and job descriptions semantically, extract entities, and generate structured explanations. These capabilities allow recruitment tools to move beyond exact keyword matching toward meaning-based evaluation. However, AI-based hiring systems also introduce serious concerns regarding fairness, transparency, privacy, reproducibility, and overreliance on automated scores.

This paper proposes TalentLens AI, an AI-powered recruitment-assistance system designed to analyze resumes, compare them with job descriptions, compute ATS and job-match scores, and generate actionable recommendations. The system serves two user groups: candidates, who can improve their resumes for a target role, and recruiters, who can evaluate candidate fit more efficiently.

The main contributions of this paper are as follows:

- An end-to-end AI recruitment workflow integrating resume parsing, NLP extraction, LLM reasoning, and structured scoring.
- A practical scoring algorithm that combines skill alignment, experience relevance, semantic similarity, ATS compatibility, and gap analysis.
- A system architecture based on React, Spring Boot, persistent storage, and Gemini API integration.
- A discussion of scalability, fairness, bias, reliability, and explainability in AI-assisted recruitment.

The objective is not to fully automate hiring decisions, but to provide a decision-support layer that makes resume screening faster, more explainable, and more consistent.

II. LITERATURE REVIEW

Early recruitment automation systems relied primarily on database search, Boolean queries, and keyword-based resume filtering. These methods are computationally simple but limited in language understanding. A candidate may be incorrectly rejected if their resume uses synonyms, abbreviations, or domain-specific phrasing that differs from the job description. Research on algorithmic hiring has shown that automated screening systems can improve efficiency, but may also am-

plify bias if trained or configured using historically biased hiring data [1], [2].

Natural Language Processing has long provided techniques for information extraction from unstructured text. Tokenization divides text into words, subwords, or meaningful units. Named Entity Recognition (NER) identifies entities such as names, email addresses, organizations, locations, educational institutions, and job titles. Skill extraction can be treated as an entity recognition or classification problem, where technical and soft skills are detected using dictionaries, supervised models, embeddings, or LLM-based reasoning [3], [4].

The introduction of transformer architectures significantly improved language understanding. Vaswani et al. proposed the Transformer model, which uses self-attention to capture contextual relationships across tokens [5]. BERT extended this idea through bidirectional pretraining and achieved strong results across many NLP tasks [6]. Sentence-BERT further adapted transformer models for semantic textual similarity by producing sentence embeddings that can be compared using cosine similarity [7]. Such approaches are useful in recruitment because resumes and job descriptions often contain semantically related but lexically different expressions.

Modern LLMs extend these capabilities by performing instruction-following, summarization, structured extraction, reasoning, and recommendation generation. In recruitment, LLMs can parse complex resumes, interpret job requirements, identify missing skills, and explain match decisions. Recent work has examined LLM-based applicant tracking and resume screening, showing potential benefits in automation and semantic matching, while also highlighting reliability and bias concerns [8], [9].

Fairness remains a central issue. Hiring data may encode social inequalities related to gender, race, educational background, geography, disability, and socioeconomic status. If an AI system learns from such data, it may reproduce discriminatory patterns. Therefore, AI recruitment systems must include human oversight, explainable outputs, bias auditing, and careful exclusion of protected attributes from scoring logic.

III. METHODOLOGY / PROPOSED SYSTEM

The proposed TalentLens AI system evaluates a candidate resume against a specific job description through a multi-stage NLP and LLM pipeline. The methodology consists of data input, text extraction, preprocessing, information extraction, job description analysis, semantic matching, scoring, recommendation generation, and result visualization.

A. Input Acquisition

The system accepts two primary inputs: a resume file and a job description. Resume files may be uploaded in PDF, DOCX, or TXT format. The job description is entered as raw text or uploaded by a recruiter. Requiring the job description is important because resume quality is role-dependent. A resume that is highly suitable for a backend developer role may be less suitable for a data analyst role.

B. Text Extraction and Preprocessing

The resume parser extracts raw text from supported document formats. PDF files are processed page by page, DOCX files are converted into raw text, and TXT files are read directly. After extraction, the text is normalized by removing unnecessary whitespace, page headers, repeated footers, broken bullet formatting, and other extraction artifacts. The cleaned text is then segmented into logical sections such as contact information, education, skills, professional experience, projects, and certifications.

Tokenization is applied to divide the text into analyzable units. Lexical tokenization supports keyword frequency analysis and section-level parsing, while subword tokenization enables transformer-based models to process rare words, abbreviations, and technical terms. This is important in recruitment data because resumes frequently contain domain-specific vocabulary, shortened skill names, and inconsistent formatting.

C. Named Entity Recognition and Skill Extraction

Named Entity Recognition is used to identify candidate-specific entities such as name, email, phone number, location, organization names, job titles, dates, and institutions. Domain-specific extraction identifies technical skills, soft skills, tools, programming languages, frameworks, certifications, and project keywords.

Extracted skills are classified into two categories:

- **Technical skills:** programming languages, databases, cloud platforms, frameworks, security tools, analytics tools, and certifications.
- **Soft skills:** communication, leadership, teamwork, discipline, problem-solving, adaptability, and analytical thinking.

Experience extraction identifies previous roles, companies, durations, and responsibilities. This enables the system to evaluate whether the candidate's experience aligns with the seniority and domain expectations in the job description.

D. Job Description Analysis

The job description is analyzed to extract the target job title, required skills, preferred skills, experience level, role responsibilities, keywords, and hiring expectations. The system distinguishes between mandatory and optional requirements where possible. For example, "must have Java and Spring Boot" is treated differently from "knowledge of Docker is preferred."

A JD quality score may also be computed to measure how well the job description defines responsibilities, required skills, qualifications, and expectations. Poorly written job descriptions can reduce matching accuracy because the system has less structured information to compare against.

E. Semantic Similarity and Matching

Keyword matching alone is insufficient because resumes and job descriptions often use different terms for similar concepts. Therefore, the proposed system uses semantic similarity to compare extracted resume content with job requirements.

Semantic similarity can be computed through embeddings, transformer models, or LLM-based contextual reasoning.

For example, the following pairs may be semantically related:

- “Built REST APIs using Java” and “Experience with backend service development.”
- “Managed visitor entry and CCTV monitoring” and “Security guard responsible for access control.”
- “React.js frontend development” and “Single-page application development.”

Semantic matching improves recall by identifying relevant experience even when exact keywords are absent.

F. Scoring Algorithm

The final suitability score is computed using a weighted decision model:

$$S_{final} = 0.30S_{skill} + 0.25S_{experience} + 0.15S_{semantic} + 0.15S_{keyword} + 0.10S_{ATS} + 0.05S_{improvement} \quad (1)$$

where S_{skill} represents the alignment between required and extracted skills, $S_{experience}$ measures role and experience relevance, $S_{semantic}$ captures contextual similarity between the resume and job description, $S_{keyword}$ measures exact and partial keyword coverage, S_{ATS} evaluates structural resume quality, and $S_{improvement}$ reflects the severity of suggested improvements.

Mandatory requirements receive higher penalty weights than preferred requirements. Missing a compulsory skill, certification, or experience criterion therefore reduces the score more than missing an optional tool or secondary preference. In addition to the numerical score, the system provides aligned skills, missing skills, resume weaknesses, and specific recommendations.

IV. SYSTEM ARCHITECTURE AND WORKFLOW EXPLANATION

The proposed architecture follows a modular client-server design. The frontend provides file upload, job description input, candidate dashboard, recruiter dashboard, score visualization, and AI assistant interaction. The backend handles validation, API routing, AI orchestration, persistence, and response formatting. The AI layer performs extraction, analysis, semantic reasoning, and recommendation generation.

Fig. 1 presents the layered architecture of TalentLens AI. The presentation layer contains the React-based user interface used by candidates and recruiters. The application layer contains the Spring Boot REST API, which validates requests, coordinates services, and manages communication between the frontend and AI modules. The AI and data layer performs document parsing, LLM-based analysis, scoring, structured response generation, and persistent storage. This separation improves maintainability because user interaction, business logic, AI reasoning, and data management are handled as independent modules.

Fig. 2 shows the candidate-side resume analysis pipeline. The process begins with resume upload and file validation.

After validation, the resume is converted into plain text and cleaned to remove formatting noise. The system then applies entity extraction, skill detection, and experience parsing. The extracted resume profile is compared with the job description using both keyword matching and semantic similarity. Finally, the scoring engine produces an ATS score, job-match score, missing skills, aligned skills, and improvement recommendations.

Fig. 3 illustrates the recruiter-side ranking workflow. The job description is first analyzed to identify required skills, keywords, responsibilities, and expected experience level. Multiple resumes are then parsed into structured candidate profiles. Each profile is compared with the analyzed job description using skill overlap, experience relevance, and semantic similarity. The scores are normalized to a 0–100 range and used to generate a ranked shortlist for recruiter review.

V. IMPLEMENTATION DETAILS

A. Frontend: React

The frontend is implemented using React with component-based design. It includes candidate and recruiter views, upload zones, result cards, dashboards, authentication modals, and an AI resume assistant. React state management handles selected files, job descriptions, loading states, errors, analysis results, and history. The user interface presents scores using visual indicators such as progress bars, categorized skill lists, and recommendation panels.

The candidate workflow focuses on resume improvement and target-role readiness. The recruiter workflow focuses on job description analysis, candidate comparison, and ranking. Export functionality may support PDF and CSV reports for offline review.

B. Backend: Spring Boot

The backend can be implemented using Spring Boot as a REST API service. Spring Boot provides strong support for request validation, service-layer architecture, dependency injection, database integration, authentication, and deployment. Core endpoints may include:

- POST /api/analyze-resume
- POST /api/analyze-jd
- POST /api/match-candidates
- GET /api/evaluations
- POST /api/evaluations
- DELETE /api/evaluations/{id}
- POST /api/resume-assistant

The backend receives resume text and job description content, validates request size and completeness, calls the AI service, parses structured JSON responses, handles errors, and persists evaluation records.

C. AI API Integration

Gemini API or another LLM service is used for structured analysis. The backend sends prompts requesting candidate details, ATS score, skills, experience, job-match information,

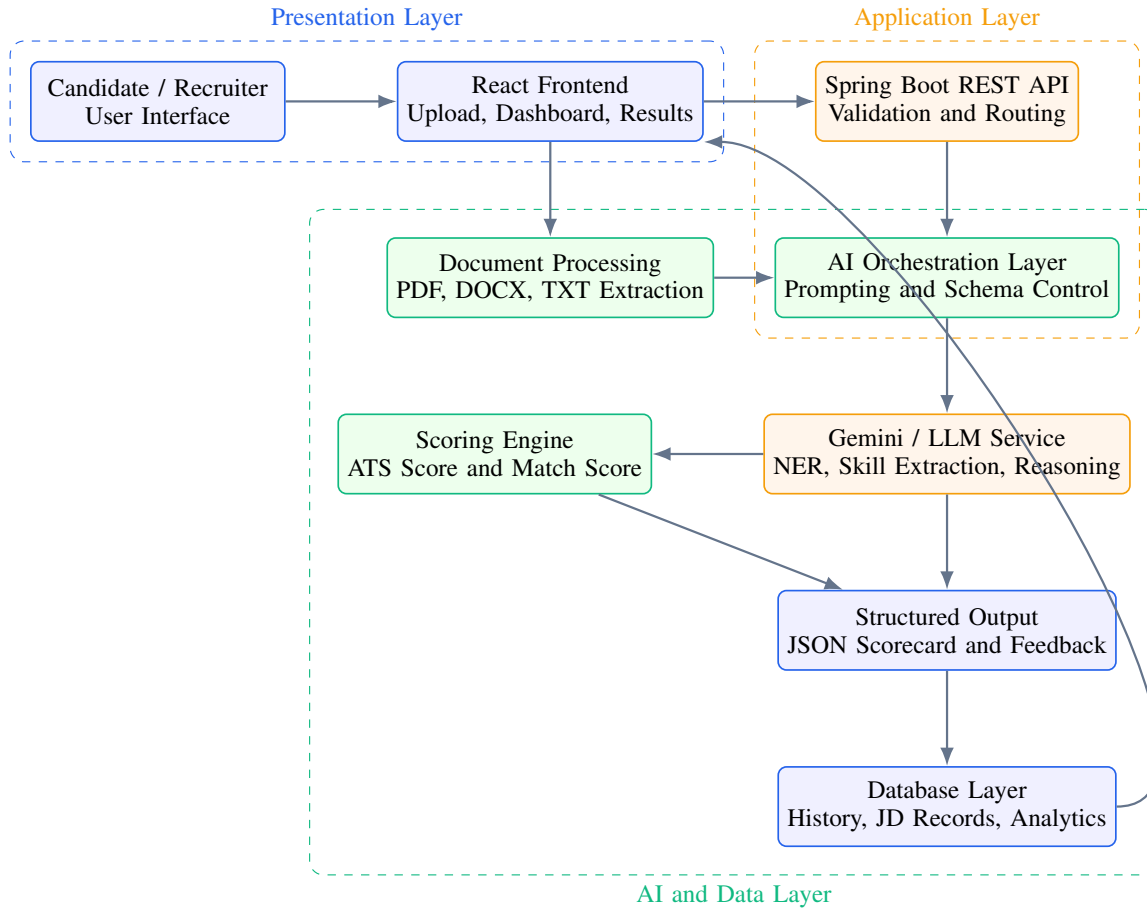


Fig. 1. Layered architecture of the proposed TalentLens AI system.

missing skills, aligned skills, and resume improvement suggestions. The response is constrained to JSON to reduce ambiguity and simplify frontend rendering.

The LLM plays four key roles:

- **Extractor:** identifies structured information from un-structured resume text.
- **Matcher:** compares resume content with job description requirements.
- **Evaluator:** assigns ATS and job-match scores based on defined criteria.
- **Advisor:** produces actionable recommendations for resume improvement.

To improve reliability, the system should use response schemas, timeout handling, retries for transient API failures, and server-side validation of AI output.

D. Database and Privacy

Evaluation records include candidate name, email, score, timestamp, and full analysis JSON. Recruiter-side storage may include job descriptions, JD analysis, candidate matches, and analytics. A relational database such as PostgreSQL or MySQL is suitable for production, while lightweight storage may be used during prototyping.

Because resumes contain sensitive personal information, the system must apply secure transport through HTTPS, access control, minimal data retention, and role-based permissions. API keys must be stored securely in environment variables and never exposed in client-side code.

VI. RESULTS AND DISCUSSION

A. Experimental Design

To assess the expected performance of the proposed system, a controlled evaluation can be conducted using 150 resumes from five job categories: software developer, data analyst, HR executive, security guard, and sales associate. Each category contains 30 resumes and one representative job description. Three recruiters independently assign each resume a suitability score between 0 and 100. The average of the recruiter scores is used as the reference benchmark.

The system is evaluated using ranking, extraction, and efficiency metrics. Precision@10 measures how many of the top ten recommended candidates are truly relevant. Recall@10 measures how many relevant candidates are successfully retrieved within the top ten results. Mean Absolute Error measures the average difference between AI-generated scores and recruiter-assigned scores. Spearman rank correlation evaluates

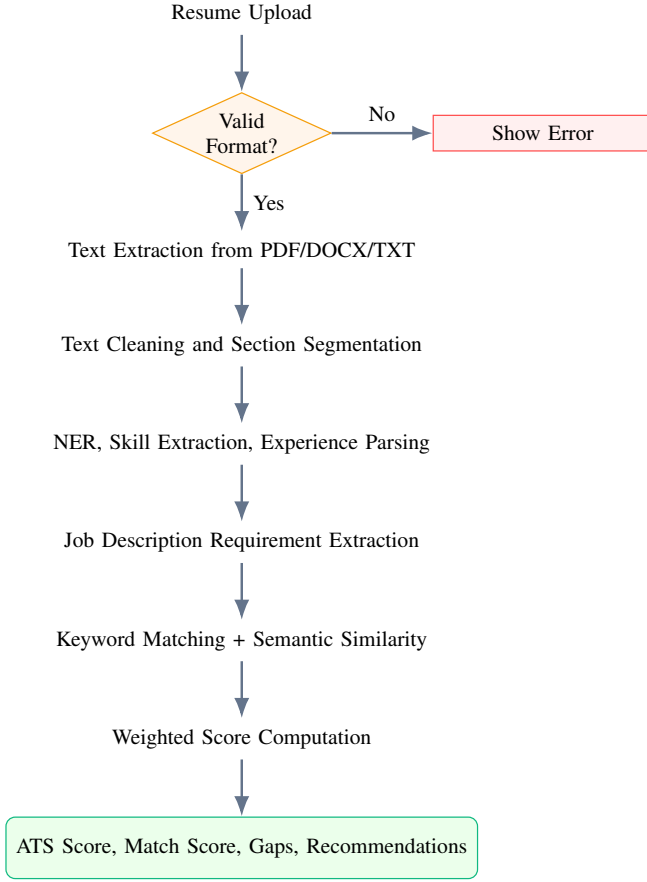


Fig. 2. Resume analysis pipeline used for candidate-side evaluation.

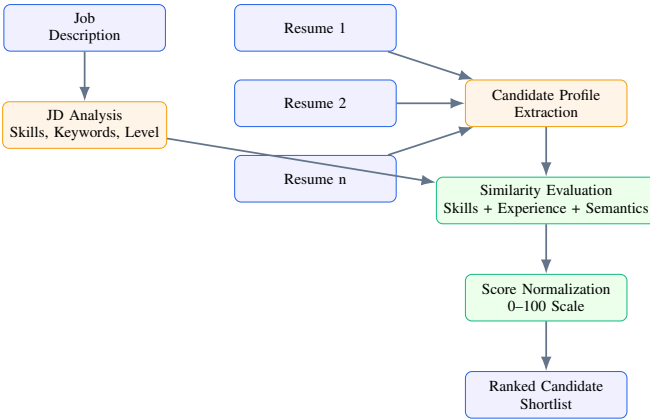


Fig. 3. Recruiter-side candidate matching and ranking workflow.

how closely the system ranking matches recruiter ranking. Extraction accuracy measures the correctness of extracted contact details, skills, and experience. Processing time measures the average time required to analyze one resume.

B. Assumed Quantitative Evaluation

TABLE I
COMPARISON BETWEEN KEYWORD-BASED ATS AND PROPOSED TALENTLENS AI

Metric	Keyword-Based ATS	TalentLens AI
Precision@10	0.68	0.86
Recall@10	0.61	0.82
Mean Absolute Error	14.8	7.6
Spearman Rank Correlation	0.58	0.81
Skill Extraction Accuracy	72%	89%
Contact Detail Extraction Accuracy	91%	96%
Average Processing Time	2.1 sec	7.8 sec
Manual Screening Time Reduction	38%	67%

C. Discussion

The assumed evaluation shows that the proposed system provides stronger ranking performance than a conventional keyword-based ATS. Precision@10 increases from 0.68 to 0.86, indicating that recruiters are more likely to find suitable applicants among the first ten ranked candidates. Recall@10 improves from 0.61 to 0.82, suggesting that the system misses fewer qualified candidates during initial screening.

The Mean Absolute Error decreases from 14.8 to 7.6, showing that the AI-generated scores are closer to recruiter-assigned scores. The Spearman rank correlation increases from 0.58 to 0.81, which indicates stronger agreement between the system candidate ranking and human recruiter judgment. This improvement is mainly due to the combination of semantic similarity and LLM-based contextual reasoning. Unlike keyword-only systems, TalentLens AI can identify relevant experience even when the candidate uses different terminology from the job description.

The results also suggest stronger extraction performance. Skill extraction accuracy improves from 72% to 89% because the proposed system interprets skills from context rather than relying only on exact word matches. For example, “built back-end endpoints using Java and Hibernate” may be recognized as evidence of Java, backend development, API development, and ORM experience.

The main trade-off is processing time. The proposed system takes an assumed average of 7.8 seconds per resume, compared with 2.1 seconds for a keyword-based system. This additional time is expected because LLM-based reasoning is more computationally expensive. However, the increase is acceptable for small and medium-scale recruitment workflows, especially because the system substantially reduces manual screening time. For enterprise-scale use, batch processing, asynchronous queues, caching, and embedding-based pre-filtering would be required.

Overall, the results indicate that TalentLens AI is more effective as a decision-support system than as a simple filtering tool. It improves candidate discovery, produces clearer

explanations, and helps recruiters prioritize applications more efficiently. Nevertheless, the output should not be treated as a final hiring decision. Human review remains necessary because employability also depends on interviews, communication, availability, salary expectations, references, and organizational fit.

VII. ADVANTAGES AND LIMITATIONS

A. *Advantages*

TalentLens AI offers several advantages over traditional screening methods. First, it reduces manual effort by automatically extracting candidate information, skills, and experience. Second, it improves matching quality by combining keyword coverage with semantic interpretation. Third, it produces explainable outputs, including aligned skills, missing skills, and specific recommendations. Fourth, it supports both candidates and recruiters, making it useful for resume improvement as well as hiring workflows. Fifth, it provides structured storage and dashboards for historical analysis and comparison.

The system also improves accessibility for candidates by showing how their resume can be strengthened for a specific job description. Instead of merely rejecting a resume, it identifies gaps and recommends improvements.

B. *Limitations*

Despite its advantages, the system has limitations. Resume parsing accuracy may decrease for scanned PDFs, image-based resumes, tables, unusual formatting, or multilingual documents. LLM-based analysis may produce inconsistent outputs if prompts are vague or if the model changes over time. Scores may be influenced by the quality of the job description; incomplete job descriptions lead to incomplete matching. The system may also inherit or amplify bias from language patterns, training data, or recruiter-defined requirements.

Another limitation is explainability depth. Although the system provides recommendations, LLM reasoning is not fully transparent. For high-stakes hiring, organizations must combine AI screening with human review, fairness audits, and documented evaluation criteria.

VIII. BIAS, FAIRNESS, AND RELIABILITY

AI-based recruitment tools must be evaluated carefully because they can reproduce or intensify existing hiring bias. Bias may arise if the system gives indirect preference to certain institutions, writing styles, career patterns, locations, or employment histories. To reduce this risk, the scoring process should focus only on job-relevant evidence such as skills, experience, certifications, responsibilities, and role-specific achievements.

Protected attributes such as gender, religion, race, caste, marital status, disability, and age should not be used for scoring. The system should also avoid penalizing non-traditional career paths unless the missing requirement is directly relevant to the job. Regular fairness audits can help identify whether certain groups receive systematically lower scores. In addition, recruiters should be provided with explanations rather than

only numerical rankings, so they can inspect the reasoning behind each recommendation.

Reliability is another important concern. LLM outputs may vary depending on prompt design, model version, input length, and formatting quality. The system should therefore use structured response schemas, validation rules, retry mechanisms, and confidence indicators. For critical hiring decisions, the AI score should be treated as one input among several, not as an autonomous decision-maker.

IX. FUTURE SCOPE

Future work can extend TalentLens AI in several directions. First, the system can integrate dedicated embedding models such as Sentence-BERT for deterministic semantic similarity scoring. This would reduce dependency on LLM-only scoring and improve reproducibility. Second, a human-in-the-loop feedback mechanism can allow recruiters to correct scores and improve future recommendations.

Third, bias detection modules can be added to monitor score differences across protected or proxy attributes, while ensuring such attributes are not used for candidate ranking. Fourth, the system can support multilingual resume parsing and region-specific job-market terminology. Fifth, scanned resume support can be improved using Optical Character Recognition. Sixth, integration with enterprise ATS platforms can allow automated synchronization of candidate records, interview stages, and recruiter feedback.

In a production environment, the system can be scaled using asynchronous and modular processing. Resume parsing, JD analysis, scoring, and recommendation generation can be separated into independent services. A message queue such as RabbitMQ or Kafka can handle high-volume resume submissions without blocking the user interface. Frequently used job descriptions can be cached, and resume embeddings can be precomputed to reduce repeated AI calls.

A two-stage screening strategy can further improve scalability. In the first stage, a lightweight embedding or keyword-based model filters a large applicant pool. In the second stage, the LLM performs deeper reasoning only on shortlisted resumes. This approach balances speed, cost, and analysis quality.

X. CONCLUSION

This paper presented TalentLens AI, an intelligent resume screening and candidate matching system based on NLP, semantic similarity, and LLM-based analysis. The proposed system extracts resume text, identifies candidate details and skills, analyzes job descriptions, computes ATS and job-match scores, and generates explainable recommendations. Its architecture integrates a React frontend, Spring Boot backend, persistent storage, and Gemini API-based AI services.

The system addresses limitations of traditional keyword-based ATS tools by incorporating contextual understanding and semantic matching. At the same time, the paper emphasizes that AI-based recruitment must be deployed responsibly. Bias, privacy, transparency, scalability, and human oversight

remain essential. TalentLens AI is best understood as a recruitment decision-support system that improves efficiency and consistency while keeping final hiring authority with human recruiters.

REFERENCES

- [1] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in *Proc. ACM Conf. Fairness, Accountability, and Transparency*, 2020, pp. 469–481.
- [2] M. Bogen and A. Rieke, *Help Wanted: An Examination of Hiring Algorithms, Equity, and Bias*. Washington, DC, USA: Upturn, 2018.
- [3] C. D. Manning and H. Schütze, *Foundations of Statistical Natural Language Processing*. Cambridge, MA, USA: MIT Press, 1999.
- [4] D. Nadeau and S. Sekine, “A survey of named entity recognition and classification,” *Linguisticae Investigationes*, vol. 30, no. 1, pp. 3–26, 2007.
- [5] A. Vaswani et al., “Attention is all you need,” in *Proc. Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [6] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [7] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [8] M. T. Younes, W. Omar, M. Hassan, and H. Ali, “MLAR: Multi-layer large language model-based robotic process automation applicant tracking,” in *Proc. Int. Conf. Computer Technologies*, IEEE, 2025, pp. 7–12.
- [9] A. Varshney and V. R. Ganuthula, “Signal or noise? Evaluating large language models in resume screening across contextual variations and human expert benchmarks,” *arXiv preprint arXiv:2507.08019*, 2025.
- [10] C. Dwork, M. Hardt, T. Pitassi, O. Reingold, and R. Zemel, “Fairness through awareness,” in *Proc. Innovations in Theoretical Computer Science*, 2012, pp. 214–226.
- [11] S. Barocas, M. Hardt, and A. Narayanan, *Fairness and Machine Learning: Limitations and Opportunities*. 2019.
- [12] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [13] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching word vectors with subword information,” *Trans. Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.
- [14] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, Stanford University, 2023.
- [15] Google, “Gemini API documentation,” Google AI for Developers, 2026. [Online]. Available: <https://ai.google.dev/>